

TrustUs

Mobile App — Complete Build Spec (From Scratch)

Flutter — aligned exactly to the real, already-built server contract

No mobile code exists yet. This is the ground-up spec, corrected to match reality.

Table of Contents

0. Grounding Note

The uploaded project contains a real, substantially-built server (Trustus-server/) and web dashboard (Trustus-web/), but no mobile app directory at all — this is a from-scratch build. Rather than reuse the earlier, pre-build Mobile spec from this project's planning phase unchanged, this document is corrected against the real server's actual routes, state machine, and event types, confirmed by direct code inspection. Two corrections worth calling out up front: event types are DISPATCH / RECEIVE / SALE / RETURN / FLAG / UNFLAG (not manufacturer_ship / distributor_receive / sale_dispense as earlier drafts assumed), and the public verify result has three states — genuine / flagged / recalled — not four.

1. Scope

One Flutter app used by every org-type role that scans physical products: manufacturer, regional_agent, distributor_authorized, distributor_sub, retailer, transporter. social_seller's core feature (Social Listings) is primarily a web/desktop workflow (linking IDs to social posts) but should also be reachable from mobile for on-the-go listing creation — included as a secondary nav item, not a primary scanning role. This app is not the consumer verification experience — that is the existing public web route (/v/:unitId), which already works and should remain a lightweight, no-login web page, not a native app feature.

2. Tech Stack

- Flutter 3.x stable, Dart null-safety throughout.
- State management: Riverpod.
- Local database: Drift (sqlite) for the offline event queue — correctness of this queue directly determines whether field scans survive connectivity gaps, so type-safety here is worth the extra setup over raw sqlite.
- Networking: dio, with an interceptor mirroring the web app's already-proven pattern: in-memory access token, refresh token in secure storage, single-flight refresh with request queuing on concurrent 401s (see Trustus-web/src/api/client.ts for the exact reference behavior to replicate).
- QR/barcode scanning: mobile_scanner.
- GPS: geolocator.
- Push notifications: firebase_messaging, for Alerts — secondary priority, not required for first working build.

3. Project Structure

```
trustus_mobile/
  lib/
    main.dart
    core/
      api_client.dart      # dio instance, interceptors – mirror Trustus-web's
client.ts pattern exactly
      auth/                # token storage (flutter_secure_storage), refresh logic
      offline/
        event_queue_db.dart # Drift schema for pending_events
        sync_service.dart   # background sync when connectivity returns
      features/
        auth/              # OTP login
        onboarding/        # create org / accept invite – mirrors OnboardingPage.tsx
        home/              # role-based dashboard/home
        scan/              # camera scan + event-type selection + confirmation
        batches/           # manufacturer/regional_agent/distributor_authorized:
batch & unit generation
        packing/           # container aggregation – box/pallet/shipment
```

```

    history/          # unit/container event history (GET /units/:id/history)
    alerts/           # alerts list (GET /alerts)
    social/           # secondary: create a Social Listing on the go
    profile/          # account/org settings, logout – mirrors the new Web My
Profile page
  models/            # Dart classes mirroring the real API schemas exactly
(Section 5)
  widgets/
  test/
pubspec.yaml

```

4. Real API Contract to Build Against

The exact 39 real server endpoints, extracted directly from `app/api/v1/*.py` — build against these, not any earlier draft.

Resource	Endpoints Relevant to Mobile
Auth	POST /auth/otp/request · POST /auth/otp/verify · POST /auth/refresh · POST /auth/logout
Orgs	POST /orgs · GET /orgs/me · POST /orgs/{org_id}/kyc-documents · GET/PATCH /orgs/me/notification-settings
Network	POST /network/invites · POST /network/invites/{invite_code}/accept · GET /network/tree
Products & Batches	POST/GET /products · GET /batches/lookup · POST /batches · GET /batches/{batch_id} · POST /batches/{batch_id}/units · POST /batches/{batch_id}/recall
Units	GET /units/{unit_id} · GET /units/{unit_id}/history
Containers	POST /containers · GET /containers/{container_id} · POST /containers/{container_id}/units · POST /containers/{container_id}/disaggregate
Events	POST /events (the core scan-submission endpoint) · GET /events
Alerts	GET /alerts · GET/PATCH /alerts/{alert_id}
Social	POST/GET /social-listings · DELETE /social-listings/{listing_id}

5. Real Event Types & States — Build Against These Exactly

Current State	event_type to Submit	Resulting State
ISSUED / RECEIVED / RETURNED	DISPATCH	IN_TRANSIT
IN_TRANSIT	RECEIVE	RECEIVED
RECEIVED	SALE	SOLD
SOLD	RETURN	RETURNED

The Scan Screen's event-type picker (Section 6.3) must offer only the transitions valid for the unit's current state and the user's role — e.g. a Retailer scanning a RECEIVED unit should see “Sell” (SALE) and, if returns are supported at retail, “Return” (RETURN); a Distributor receiving an IN_TRANSIT unit should see only “Receive” (RECEIVE). Do not show DISPATCH as an option to a role that never ships product onward if that's not part of their real workflow — confirm the exact role-to-action mapping against how the Web dashboard's Batches/Events pages expose these same actions, so both clients agree.

6. Screens — Full Detail

6.1 Login / OTP

- Phone entry → POST /auth/otp/request; OTP entry → POST /auth/otp/verify; store tokens in flutter_secure_storage.
- If the returned user has no org_id, route to Onboarding, not Home.

6.2 Onboarding

- Create org (POST /orgs) or accept an invite (POST /network/invites/{code}/accept) — mirror the web OnboardingPage's two-path design exactly so a user switching between web and mobile has a consistent mental model.

6.3 Scan Screen — Core Feature

- Camera scan via mobile_scanner, decode the QR payload (a URL like https://trustus.app/v/{unit_id} or /verify/{unit_id} — extract the unit_id from the path, matching the real public-route pattern in App.tsx).
- Role- and state-aware event-type picker per Section 5's table.
- GPS capture via geolocator at confirmation — request permission with a rationale screen first, proceed without GPS if denied or unavailable (never block the scan), matching the production blueprint's original GPS fallback design.
- On confirm, write to the local Drift queue immediately with a client-generated client_event_id, then attempt POST /events — the server's idempotency on client_event_id (confirmed in the real code) makes offline retry-on-reconnect safe by design.

6.4 Batch & Unit Generation (manufacturer / regional_agent / distributor_authorized)

- Form → POST /batches, then POST /batches/{batch_id}/units; display the returned unit list with an export/share action for the manufacturer's label-printing workflow.
- Recall action (manufacturer org_admin only) → POST /batches/{batch_id}/recall with a required reason and a confirmation dialog.

6.5 Packing / Containers

- Repeating scan mode that accumulates Unit scans into a running list → “Finish Box” → POST /containers with container_type='box' and the accumulated child unit IDs.
- Scanning existing Containers into a Pallet/Shipment → same flow via POST /containers/{container_id}/units for adding to an existing container, or POST /containers for a new parent — confirm against the real containers.py which of these two call patterns the server expects for nesting an existing container versus adding loose units, since the route list (Section 4) shows /containers/{id}/units as the add-children endpoint.
- Disaggregate action (opening a box) → POST /containers/{container_id}/disaggregate, with a “cannot be undone” confirmation.

6.6 History

- Scan or select any Unit/Container to view GET /units/{id}/history as a timeline — same data the Web Dashboard's corrected Unit Detail page (Section 4.4 of the Web doc) now shows, so both clients present identical history.

6.7 Alerts

- List GET /alerts scoped to the user's own org; PATCH /alerts/{id} to mark reviewed if the role permits.

6.8 Social Listings (secondary nav item)

- For social_seller (and optionally any role wanting to create a listing on the go): create via POST /social-listings with platform, post URL, and scanned/entered unit IDs; list own listings via GET /social-listings; delete via DELETE /social-listings/{id}.

6.9 Profile

- Mirrors the Web Dashboard's new My Profile page (Section 4.15 of the Web doc): user_id, phone, role, org card, logout — keep both clients' account screens showing the same fields.

7. Offline-First Sync

```
class PendingEvents extends Table {
  TextColumn get clientEventId => text(); // uuid, generated locally
  TextColumn get targetId => text();
  TextColumn get eventType => text(); // DISPATCH | RECEIVE | SALE | RETURN
  DateTimeColumn get timestamp => dateTime();
  RealColumn get gpsLat => real().nullable();
  RealColumn get gpsLng => real().nullable();
  BoolColumn get synced => boolean().withDefault(const Constant(false));
  IntColumn get retryCount => integer().withDefault(const Constant(0));
  @override Set<Column> get primaryKey => {clientEventId};
}
```

- On app resume and connectivity-restored events, POST all unsynced rows oldest-first; mark synced=true on 200, never delete (keep for local History even offline).
- On failure, increment retryCount with exponential backoff — run as a background isolate, never block the UI thread.
- Because the server enforces idempotency on client_event_id (confirmed real behavior, Section 4), a retried sync after partial failure is always safe to resend without custom conflict resolution.

8. Auth Token Handling — Match the Web App's Proven Pattern Exactly

Trustus-web/src/api/client.ts already implements a correct, tested pattern: access token in memory, refresh token in secure storage (sessionStorage on web — use flutter_secure_storage on mobile), single-flight refresh with a queue for concurrent requests hitting 401 simultaneously, and a global logout event on unrecoverable refresh failure. Port this logic directly rather than re-deriving it — it's already correct and tested in production use on the web client.

9. Testing Requirements

- Widget tests for the Scan Screen: successful decode, invalid QR, camera/GPS permission-denied paths.
- Unit test for the sync service: queue an event, simulate failure then recovery, assert exactly one successful POST (no duplicates) — relies on the server's real idempotency guarantee, so this test also implicitly verifies that guarantee holds.
- Integration test for the full offline scenario: airplane-mode a scan, confirm local queue, restore connectivity, confirm sync and UI update.

10. Build & Release

- Android: request camera + location permissions with rationale dialogs; target current Play Store minimum API level.

- iOS: NSCameraUsageDescription and NSLocationWhenInUseUsageDescription with specific, non-generic descriptions.
- Flavors (dev/staging/prod) for API base URL — never hardcode; match the three real backend environments this server is designed to run in (Section 9 of the Server doc).